

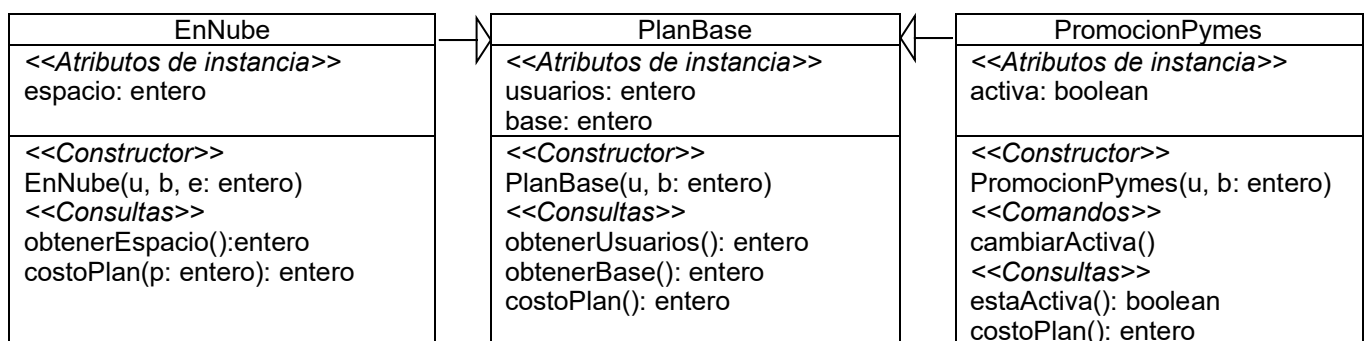


Actividades prácticas Herencia y Polimorfismo

Objetivos:

- Implementar un diagrama de clases relacionadas por herencia.
- Comprender la diferencia entre sobrecarga y redefinición.
- Conocer el concepto de polimorfismo, las diferencias entre tipo estático y dinámico de una variable y las restricciones sobre el polimorfismo que impone Java.

EJERCICIO 1. Una empresa de desarrollo de software ofrece distintos planes para suscribirse a sus productos:



En la clase *PlanBase*:

- *costoPlan(): entero*. Retorna el valor del atributo *base* multiplicado por 2, más la cantidad de *usuarios* multiplicada por 10.

En la clase *PromocionPymes* el constructor inicializa *activa* en true.

- *cambiarActiva()* modifica el valor del atributo *activa*, si está en true pasa a false y viceversa.
- *costoPlan(): entero*. Si *activa* es true retorna el valor *base*, en caso contrario retorna el valor de *base* más la cantidad de *usuarios* multiplicada por 5.

En la clase *EnNube*:

- *costoPlan(p: entero): entero*. Retorna el valor de *espacio* multiplicado por *p*, más el valor *base*, más la cantidad de *usuarios*.

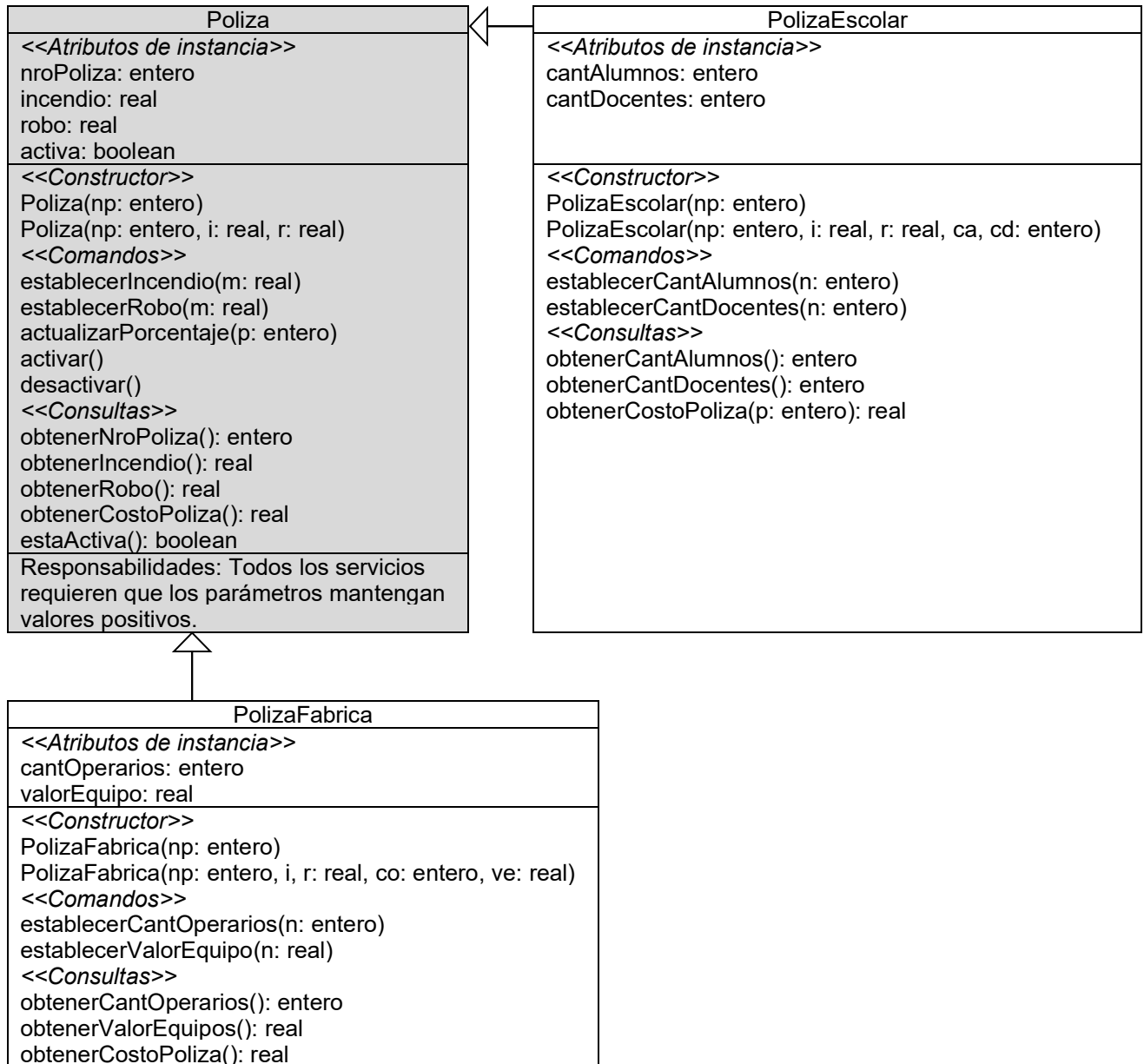
a) Implemente el conjunto de clases modeladas en el diagrama considerando que todos los atributos se declaran protegidos.

b) Muestre la salida y el diagrama de objetos luego de ejecutarse el siguiente segmento de código:

```
PlanBase b1, b2, b3;  
EnNube b4;  
b1 = new PlanBase(10000, 50);  
b2 = new PromocionPymes(1000, 50);  
b3 = new EnNube(2000, 60, 3000);  
b4 = new EnNube(1000, 50, 2500);  
System.out.println(b1.costoPlan());  
System.out.println(b2.costoPlan() + " " + b3.costoPlan());  
System.out.println(b4.costoPlan() + " " + b4.costoPlan(6));
```



EJERCICIO 2: En el siguiente diagrama las clases *PolizaEscolar* y *PolizaFabrica* extienden a la clase *Poliza* definida en un práctico anterior:



En la clase *PolizaEscolar*:

- *obtenerCostoPoliza(p: entero): real*. Se computa como el costo de cualquier póliza, más *cantAlumnos* por *p* por 1.5, más *cantDocentes* por *p*.

En la clase *PolizaFabrica*:

- *obtenerCostoPoliza(): real*. Se computa como el costo de cualquier póliza, más *cantOperarios* por 500, más *valorEquipo* por 0.8.



- a) Implemente las clases *PolizaEscolar* y *PolizaFabrica*.
- b) Muestre la salida del siguiente código definido en una clase cliente:

```
Poliza p1, p2, p3;  
p1 = new Poliza(111, 100, 120);  
p2 = new PolizaEscolar(112, 100, 120, 1000, 1500);  
p3 = new PolizaFabrica(113, 100, 120, 20, 50000);  
System.out.println(p1.obtenerNroPoliza()+" "+p1.obtenerCostoPoliza());  
System.out.println(p2.obtenerNroPoliza()+" "+p2.obtenerCostoPoliza());  
System.out.println(p3.obtenerNroPoliza()+" "+p3.obtenerCostoPoliza());
```

- c) Analice el siguiente segmento de código definido en una clase cliente:

```
PolizaEscolar p4;  
Poliza p5;  
p4 = new PolizaEscolar(114, 100, 120, 1000, 1500);  
p5 = new PolizaEscolar(115, 100, 120, 1000, 1500);  
System.out.println(p4.obtenerNroPoliza()+" "+p4.obtenerCostoPoliza(2));  
System.out.println(p5.obtenerNroPoliza()+" "+p5.obtenerCostoPoliza(2));
```

- d) Indique cuál es el tipo estático y dinámico de p1, p2, p3, p4 y p5 definidas antes.

OBSERVACIÓN: dado que los atributos de la clase *Poliza* están encapsulados, si desde la clase derivada debe acceder a los atributos de la clase padre, debe usar los servicios como cualquier clase cliente.

EJERCICIO 3: Dadas las siguientes implementaciones:

<pre>class Base{ int f(){ return 0; } int g(){ return 1; } int h(){ return 2; } }</pre>	<pre>class Hija extends Base{ int f(int x){ return 10*x; } int g(){ return 11; } }</pre>	<pre>class Nieta extends Hija{ int f(){ return 20; } int l(){ return 21; } int h(){ return 22; } }</pre>
---	--	--

Y el segmento de código en una clase cliente de las clases *Base*, *Hija* y *Nieta*:

```
Base u1, u2, u3, u4;  
Hija d1, d2;  
Nieta t;  
t = new Nieta();  
u1 = new Base();  
u2 = new Hija();  
u3 = new Nieta();  
u4 = t;  
d1 = new Hija();  
d2 = new Nieta();
```



Muestre la salida de los siguientes segmentos de código:

```
System.out.println(u1.f());
System.out.println(u2.f());
System.out.println(u3.f());
System.out.println(u4.f());
System.out.println(u1.g());
System.out.println(u2.g());
System.out.println(u3.g());
System.out.println(u4.g());
System.out.println(u1.h());
System.out.println(u2.h());
System.out.println(u3.h());
System.out.println(u4.h());
System.out.println(d1.f());
System.out.println(d2.f());
System.out.println(d1.f(-1));
System.out.println(d2.f(-1));
System.out.println(d1.g());
System.out.println(d2.g());
System.out.println(d1.h());
System.out.println(d2.h());
System.out.println(t.f());
System.out.println(t.g());
System.out.println(t.h());
System.out.println(t.f(-1));
System.out.println(t.l());
```

EJERCICIO 4: Dadas las siguientes implementaciones:

<pre>class Base{ protected int x; public Base(int p){ x = p; } int f(){ return x; } int g(){ return x*2; } int h(){ return x*3; } }</pre>	<pre>class Hija extends Base{ protected int y; public Hija(int p1,int p2){ super(p1); y = p2; } int f(int p){ return x*p; } int g(){ return x*10+y; } }</pre>	<pre>class Nieta extends Hija{ public Nieta(int p1,int p2){ super (p1,p2); } int f(){ return x*-1; } int l(){ return x*-2; } int h(){ return x*-3; } }</pre>
---	---	--

Y las siguientes declaraciones de variables:

```
Base u1, u2, u3, u4;
Hija d1, d2;
Nieta t;
```



Indique qué valores tienen los parámetros:

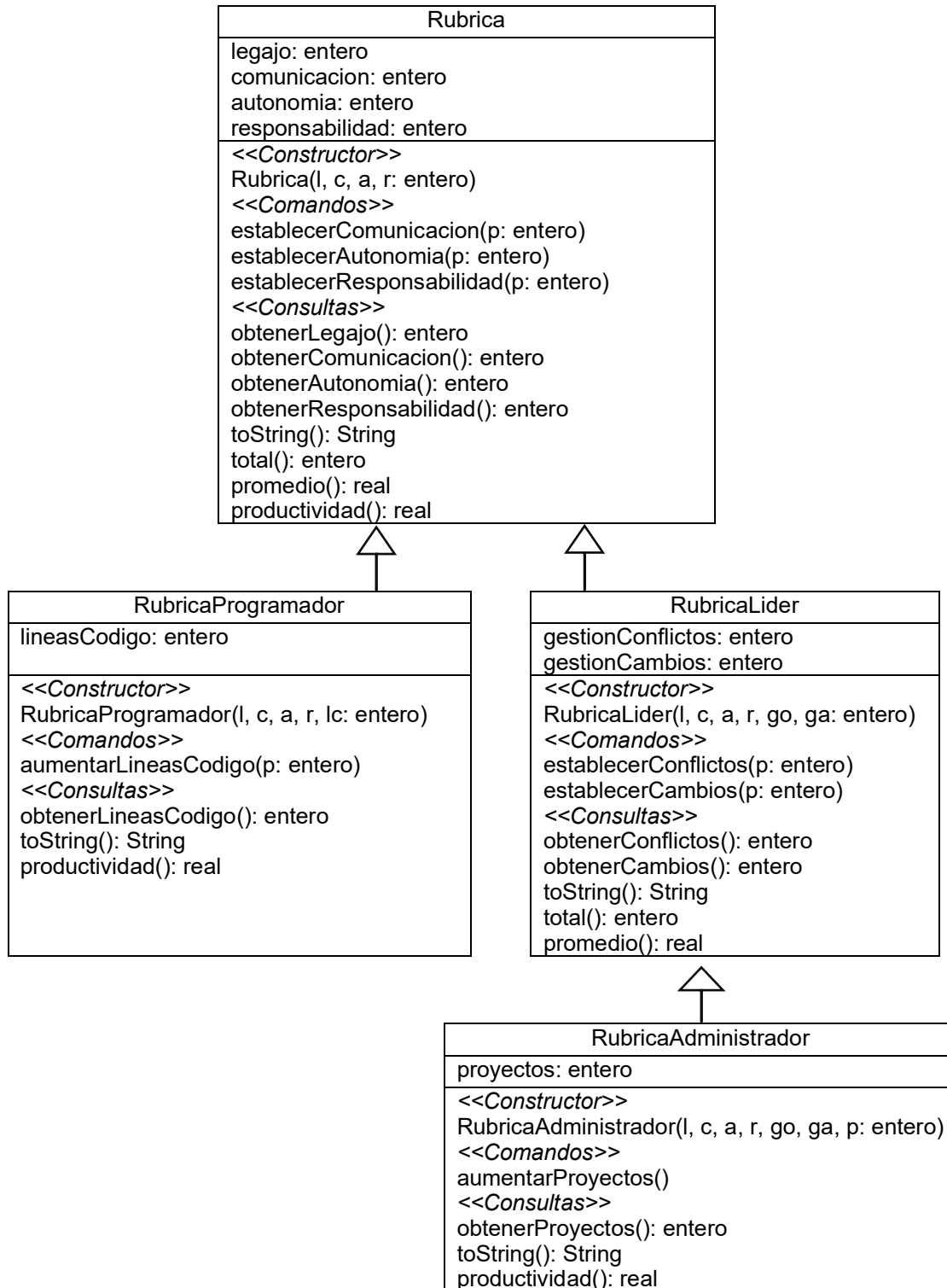
```
t = new Nieta(p1, p2);  
u1 = new Base(p3);  
u2 = new Hija(p4, p5);  
u3 = new Nieta(p6, p7);  
u4 = t;  
d1 = new Hija(p8, p9);  
d2 = new Nieta(p10, p11);
```

Si la salida del siguiente segmento de código, es el que aparece a la derecha:

System.out.println(u1.f());	2
System.out.println(u2.f());	2
System.out.println(u3.f());	-2
System.out.println(u4.f());	-2
System.out.println(u1.g());	4
System.out.println(u2.g());	25
System.out.println(u3.g());	26
System.out.println(u4.g());	23
System.out.println(u1.h());	6
System.out.println(u2.h());	6
System.out.println(u3.h());	-6
System.out.println(u4.h());	-6
System.out.println(d1.f());	3
System.out.println(d2.f());	-3
System.out.println(d1.f(-1));	-3
System.out.println(d2.f(-1));	-3
System.out.println(d1.g());	32
System.out.println(d2.g());	34
System.out.println(d1.h());	9
System.out.println(d2.h());	-9
System.out.println(t.f());	-2
System.out.println(t.g());	23
System.out.println(t.h());	-6
System.out.println(t.f(-1));	-2
System.out.println(t.l());	-4



EJERCICIO 5. Una empresa ha decidido implementar un mecanismo de evaluación por rúbrica para sus empleados. La primera versión del sistema de rúbricas ha quedado modelada por el siguiente diagrama:





En la clase *Rubrica*:

- *total()*: *entero*. Computa la suma de los atributos *comunicacion*, *autonomia* y *responsabilidad*.
- *promedio()*: *real*. Computa el promedio de los atributo *comunicacion*, *autonomía* y *responsabilidad*.
- *productividad()*: *real*. Computa 2 por *comunicación* más 3 por *autonomía* más 5 por *responsabilidad*.

En la clase *RubricaProgramador*:

- *productividad()*: *real*. Computa 1.5 por *comunicación* más 3 por *autonomía* más 5 por *responsabilidad* más (*lineasCodigo*/10000).

En la clase *RubricaLider*:

- *total()*: *entero*. Computa la suma de los atributos *comunicacion*, *autonomía*, *responsabilidad*, *gestionConflictos* y *gestionCambios*.
- *promedio()*: *real*. Computa el promedio de los atributos *comunicacion*, *autonomía*, *responsabilidad*, *gestionConflictos* y *gestionCambios*.

En la clase *RubricaAdministrador*:

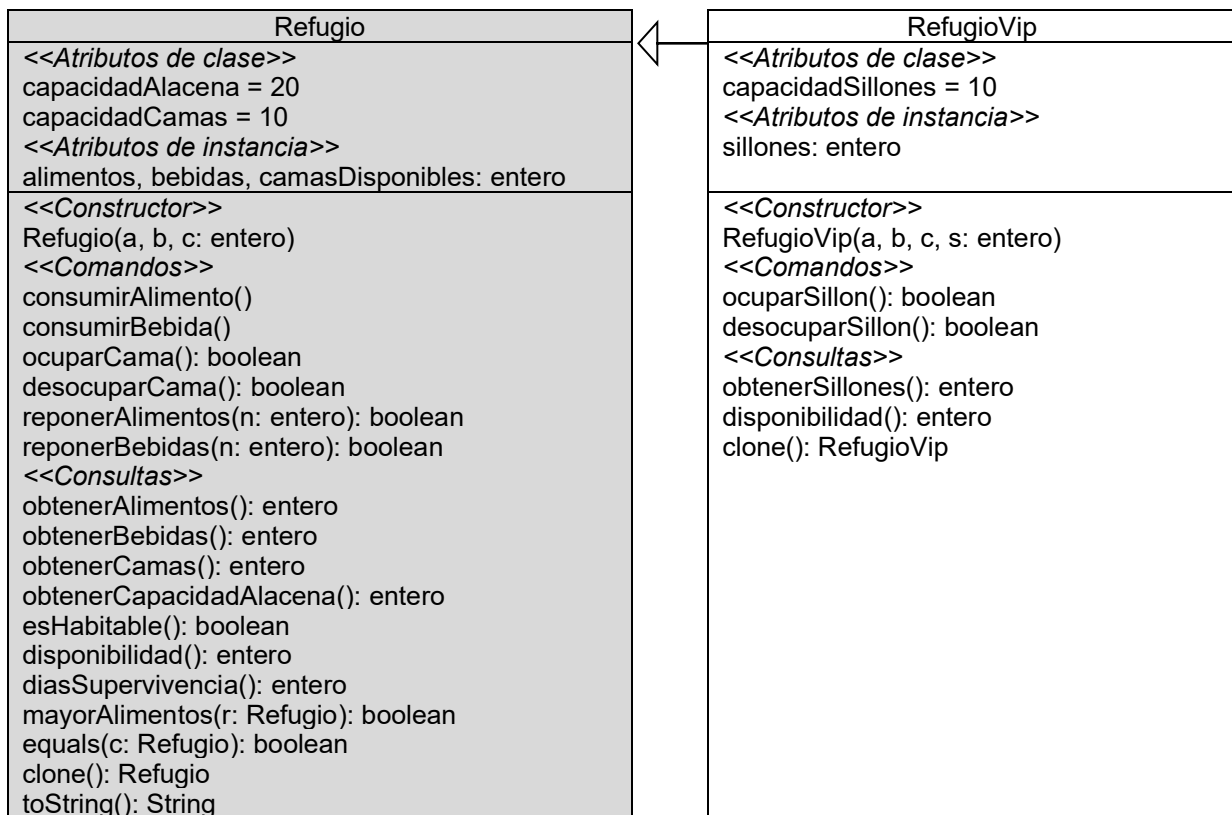
- *productividad()*: *real*. Computa 4 por su promedio más *proyectos* por 10.

- a) Implemente el modelo en Java declarando los atributos protegidos.
- b) Muestre la salida del siguiente segmento de instrucciones:

```
Rubrica r1, r2, r3, r4;
RubricaAdministrador r5;
r1 = new Rubrica(101, 10, 30, 20));
System.out.println(r1.toString());
System.out.println(r1.total()+" "+r1.promedio()+" "+r1.productividad());
r2 = new RubricaProgramador(102, 10, 30, 20, 32000);
System.out.println(r2.toString());
System.out.println(r2.total()+" "+r2.promedio()+" "+r2.productividad());
r3 = new RubricaLider(103, 10, 30, 20, 100, 100);
System.out.println(r3.toString());
System.out.println(r3.total()+" "+r3.promedio()+" "+r3.productividad());
r4 = new RubricaAdministrador(104, 10, 30, 20, 100, 100, 50);
System.out.println(r4.toString());
System.out.println(r4.total()+" "+r4.promedio()+" "+r4.productividad());
r5 = new = new RubricaAdministrador(105, 10, 30, 20, 100, 100, 52);
System.out.println(r5.toString());
System.out.println(r5.total()+" "+r5.promedio()+" "+r5.productividad());
```



EJERCICIO 6: Dado el siguiente diagrama:



En la clase *RefugioVip*

- *ocuparSillon(): boolean*. Si hay sillones disponibles, incrementa sillones en 1 y retorna true, si no retorna false.
- *desocuparSillon(): boolean*. Si hay sillones ocupados, decrementa en 1 sillones y retorna true, si no retorna false.
- *disponibilidad(): entero*. Retorna la disponibilidad de cualquier refugio más la cantidad de sillones.

Observación:

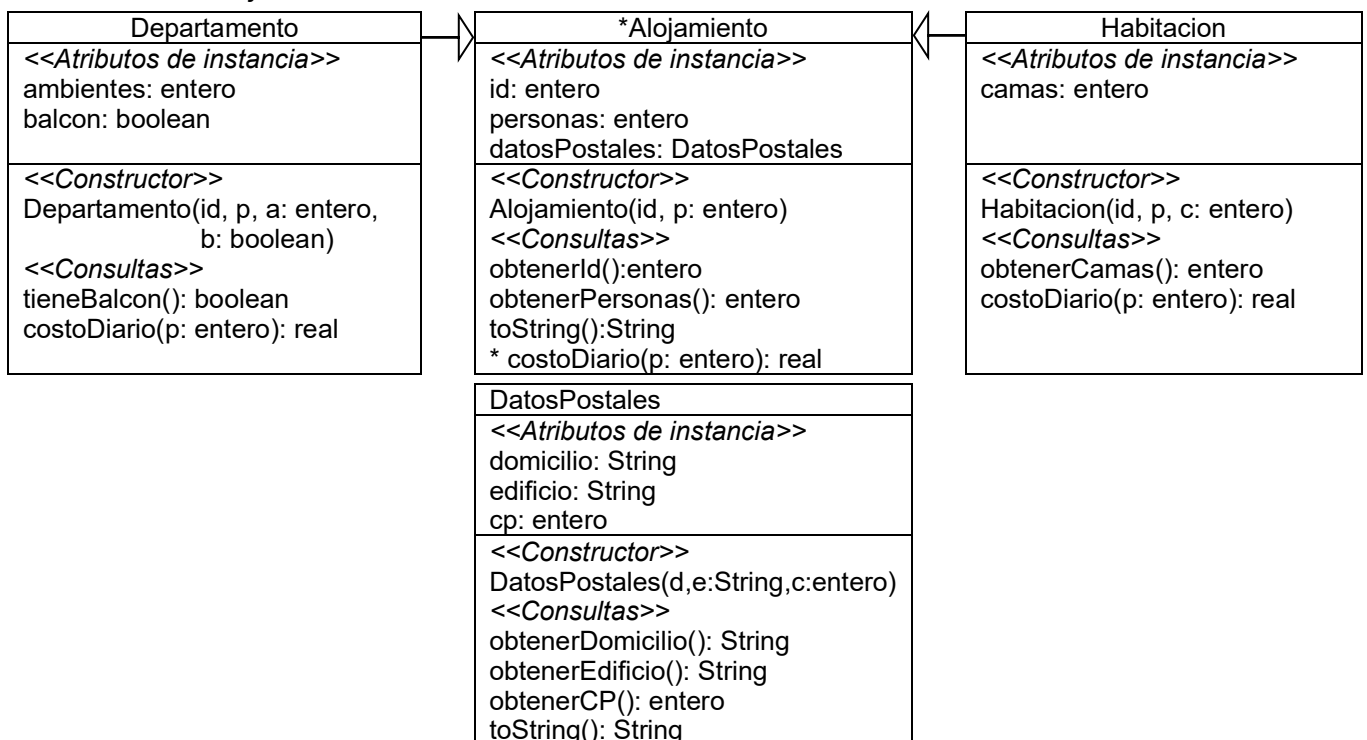
En este modelo no se redefine el método *equals*, de modo que de acuerdo a este diseño dos refugios son equivalentes si tienen la misma cantidad de alimentos, bebidas y *camasDisponibles*, aunque no tengan la misma cantidad de sillones.

- Modifique la implementación de la clase *Refugio* de modo tal que los atributos de instancia queden protegidos en lugar de ser privados.
- Implemente una clase *RefugioVip* que extiende a la clase *Refugio*
- Muestre la salida del siguiente tester



```
class testerVip{
    public static void main(String a[]){
        Refugio c1, c2, c4, clon1, clon2;
        RefugioVip c3, clon3;
        c1 = new Refugio(1,2,3);
        c2 = new RefugioVip(2,1,3,4);
        c3 = new RefugioVip(3,2,4,5);
        c4 = new RefugioVip(1,2,3,10);
        clon1 = c1.clone();
        clon2 = c2.clone();
        clon3 = c3.clone();
        System.out.println(clon1.toString());
        System.out.println(clon2.toString());
        System.out.println(clon3.toString());
        if (c1.equals(c4)){
            System.out.println("Los refugios ");
            System.out.println(c1.toString());
            System.out.println(c4.toString());
            System.out.println("serían equivalentes en este diseño");
        }
    }
}
```

EJERCICIO 7: Dado el siguiente diagrama en el cual las clases *Habitacion* y *Departamento* extienden a la clase abstracta *Alojamiento*:



En la clase *Departamento*:

- *costoDiario(p: entero): real*. Computa la cantidad de *ambientes* por *p* y suma 150 si tiene *balcón*.

En la clase *Habitacion*:

- *costoDiario(p: entero): real*. Computa la cantidad de *camas* por *p/2*.
a) Implemente el diagrama.



- b) Implemente una clase tester y dibuje el diagrama de objetos para una instancia de *Habitacion* y una de *Departamento*.

Implemente todo servicio trivial que considere necesario.

EJERCICIO 8. Dada la instrucción "**C v = new D();**" donde *D* es una clase derivada de *C*, indique cuáles de las siguientes afirmaciones son correctas:

- a) El constructor de la clase *D* debe incluir una instrucción *super* que invoca al constructor de la clase *C*.
- b) *C* es el tipo estático de la variable y determina los mensajes que el objeto ligado a la variable *v* puede recibir.
- c) *D* puede redefinir cualquiera de los métodos provistos por *C*, excepto aquellos que se definan con el modificador "*final*".
- d) Los métodos de la clase *D* pueden acceder a todos los atributos y servicios definidos en *C*.

EJERCICIO 9. Dadas las siguientes implementaciones:

```
class Alfa{
    protected int n;
    protected String s;
    public Alfa(){
        n = 10; s = new String("Alfa");    }
    public String p(int x){
        return s + " p n:" + n + " x: " + x;    }
    public String q(int x){
        return s + " q n:" + n + " x: " + x;    }
    public String r(int x){
        return s + " r n:" + n + " x: " + x;    }
    public String v(char x){
        return s + " v n:" + n + " x: " + x;    }
}
```

```
class Beta extends Alfa{
    protected int m;
    protected String ss;
    public Beta(){
        super();
        m = 11;
        ss = new String ("Beta"); }
    public String p(int x){
        return ss+" p m: "+m+" x: "+x;    }
    public String q(String x){
        return ss+" q m: "+m+" x: "+x;    }
}
```

```
class Delta extends Beta{
    protected int l;
    protected char ch;
    public Delta(){
        super();
        l = 12;
        ch = 'D';}
    public String q(int x){
        return ch+" q l: "+l+" x: "+x;    }
    public String t(int x){
        return ch+" t l: "+l+" x: "+x;    }
    public String v(int x){
        String sss= ch+" v l:"+l+" x: "+x;
        return sss;
    }
}
```



Y el siguiente segmento de código en una clase cliente de *Alfa*, *Beta* y *Delta*:

```
Alfa a1 = new Alfa();
Alfa a2 = new Beta();
Beta a3 = new Beta();
Alfa a4 = new Delta();
Beta a5 = new Delta();
Delta a6 = new Delta();
```

Indique cuáles de las siguientes instrucciones provocarán errores de compilación y de ejecución. Siempre que sea posible, muestre cuál sería la salida por consola.

<pre>System.out.println(a1.p(5)); System.out.println(a2.p(5)); System.out.println(a3.p(5)); System.out.println(a4.p(5)); System.out.println(a5.p(5)); System.out.println(a6.p(5));</pre>	<pre>System.out.println(a1.q(6)); System.out.println(a2.q(6)); System.out.println(a3.q(6)); System.out.println(a4.q(6)); System.out.println(a5.q(6)); System.out.println(a6.q(6));</pre>
<pre>System.out.println(a1.q("q8")); System.out.println(a2.q("q8")); System.out.println(a3.q("q8")); System.out.println(a4.q("q8")); System.out.println(a5.q("q8")); System.out.println(a6.q("q8"));</pre>	<pre>System.out.println(a1.t(9)); System.out.println(a2.t(9)); System.out.println(a3.t(9)); System.out.println(a4.t(9)); System.out.println(a5.t(9)); System.out.println(a6.t(9));</pre>
<pre>System.out.println(a1.r(7)); System.out.println(a2.r(7)); System.out.println(a3.r(7)); System.out.println(a4.r(7)); System.out.println(a5.r(7)); System.out.println(a6.r(7));</pre>	<pre>System.out.println(a1.v(7)); System.out.println(a2.v(7)); System.out.println(a3.v(7)); System.out.println(a4.v(7)); System.out.println(a5.v(7)); System.out.println(a6.v(7));</pre>

EJERCICIO 10. Dadas las siguientes definiciones de clases:

<pre>class Uno{ int f(){ return 2; } int g(){ return 4; } int h(){ return 8; } int m(int i){ return -i; } }</pre>	<pre>class Dos extends Uno{ int f(){ return 11; } int g(){ return super.g()*3; } int l(){ return super.f()*-1 ; } int n(){ return f() *-5 ; } int m(String s){ return s.length(); } }</pre>	<pre>class Tres extends Dos{ int f(){ return 11; } int h(){ return 111; } int p(){ return 1111; } }</pre>
---	---	---



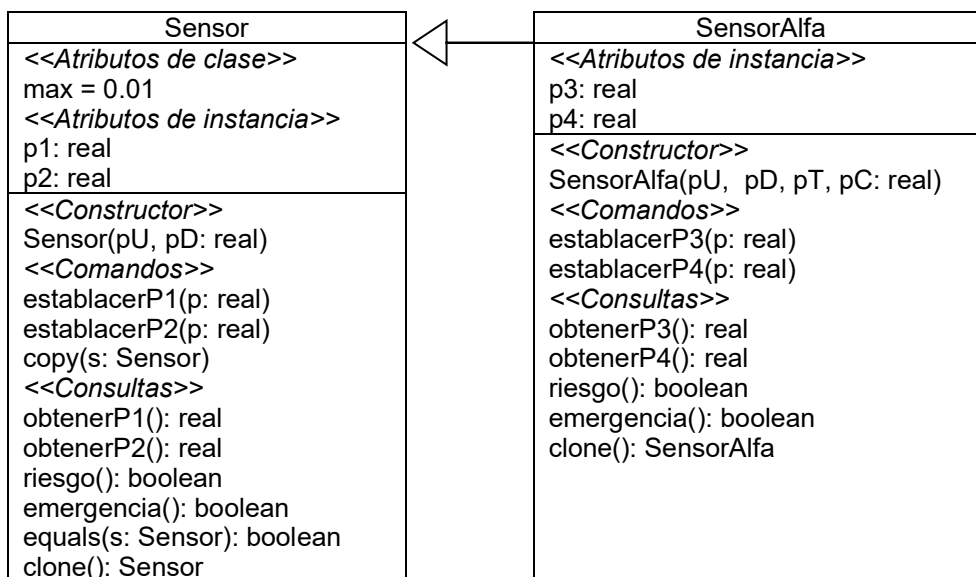
Y las siguientes instrucciones en la clase cliente:

```
Uno u1, u2, u3, u4;  
Dos d1, d2, d3;  
Tres t1, t2;  
u1 = new Uno();  
u2 = new Dos();  
u3 = new Tres();  
t1 = new Tres();  
t2 = (Tres) u3;  
u4 = t1;  
d1 = new Dos();  
d2 = new Tres();  
d3 = t2;
```

Indique qué instrucciones darán error de compilación y muestre la salida de las instrucciones válidas:

```
System.out.println("f "+u1.f()+" "+ u2.f()+" "+ u3.f()+" "+ u4.f());  
System.out.println("g "+u1.g()+" "+ u2.g()+" "+ u3.g()+" "+ u4.g());  
System.out.println("h "+u1.h()+" "+ u2.h()+" "+ u3.h()+" "+ u4.h());  
System.out.println("m "+u1.m(8)+" "+ u2.m(8)+" "+ u3.m(8)+" "+ u4.m(8));  
System.out.println("f "+d1.f()+" "+ d2.f()+" "+ d3.f());  
System.out.println("g "+d1.g()+" "+ d2.g());  
System.out.println("h "+d1.h()+" "+ d2.h());  
System.out.println("m "+d1.m(20)+" "+ d2.m(20));  
System.out.println(" f "+ t1.f()+" "+t2.f());  
System.out.println(" g "+ t1.g()+" "+t2.g());  
System.out.println(" h "+ t1.h()+" "+t2.h());  
System.out.println(" m "+ t1.m(3)+" "+t2.m(3));  
System.out.println("l "+u2.l()+" "+ u4.l());  
System.out.println("l "+d1.l()+" "+ d2.l());  
System.out.println("n "+d1.n()+" "+ d2.n());
```

EJERCICIO 11. Dado el siguiente diagrama de clases:





En la clase *SensorAlfa*

- *riesgo()*: *boolean*. Retorna true si *p2* es mayor a *p1* o bien *p4* es mayor a *p3*.
- *emergencia()*: *boolean*. Se considera en emergencia cuando *p4* es menor a *max*.

a) Implemente la clase *SensorAlfa*.

b) Muestre la salida del siguiente segmento de código de la clase tester:

```
Sensor s1 = new Sensor(1, 2);  
Sensor s2 = new SensorAlfa(1, 2, 3, 4);  
SensorAlfa s3 = new SensorAlfa(1, 2, 3, 4);  
System.out.println(s1.equals(s2));  
System.out.println(s1.equals(s3));  
System.out.println(s2.equals(s1));  
System.out.println(s2.equals(s3));  
System.out.println(s3.equals(s1));  
System.out.println(s3.equals(s2));
```

c) Analizando la salida del tester, ¿qué condiciones deben verificarse para que dos sensores sean equivalentes, de acuerdo a este diseño e implementación?

d) Agregue la siguiente implementación para el método *equals* en la clase *SensorAlfa*:

```
public boolean equals(SensorAlfa e){  
    boolean ig;  
    if (this == e)  
        ig = true;  
    else if (e == null)  
        ig = false;  
    else if (getClass() != e.getClass())  
        ig = false;  
    else {  
        SensorAlfa r = (SensorAlfa) e;  
        ig = super.equals(e) && p3 == r.obtenerP3();  
    }  
    return ig;  
}
```

e) Muestre la salida del segmento de código de la clase tester, luego de haber implementado *equals* en la clase *SensorAlfa*.

f) Modifique la signatura del método *equals* en la clase *SensorAlfa* y vuelva a mostrar la salida al ejecutar el código del tester

```
public boolean equals(Sensor e)
```

g) Analice si se verifican las propiedades de la equivalencia en cada diseño e implementación de *equals* en la clase *SensorAlfa*.

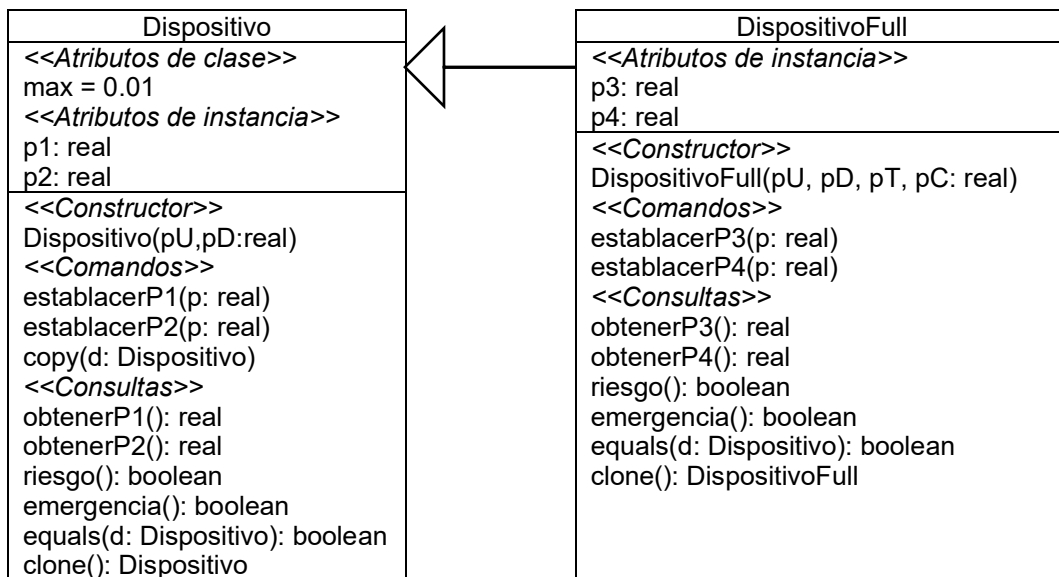
h) Modifique el método *equals* en la clase *Sensor*, manteniendo la signatura, de modo tal que dos sensores son equivalentes si tienen la misma estructura y valores en el estado interno.



EJERCICIO 12. Analice si las siguientes afirmaciones son correctas:

- Una clase abstracta puede no tener métodos abstractos.
- Una clase que extiende a una clase abstracta debe implementar todos los métodos abstractos heredados.
- Un método p definido en una clase A se dice sobrecargado si existe una clase B que hereda de A y brinda un método p con el mismo número y tipo de parámetros que A .
- El ambiente de referenciamiento de una clase D que extiende a una clase C , incluye a todos los atributos de instancia declarados en D y en C .
- Los métodos privados no pueden ser redefinidos.

EJERCICIO 13. Dado el siguiente diagrama de clases:



En la clase *Dispositivo*

- equals(d: Dispositivo): boolean*. Dos instancias de *Dispositivo* son equivalentes si tienen el mismo estado interno.
- riesgo(): boolean*. Retorna true si *p2* es mayor que *p1*.
- emergencia(): boolean*. Retorna true si *p1* es menor a *max*.

De modo que *equals* se implementa como sigue

```
public boolean equals(Dispositivo d){
    boolean ig;
    ig = p1 == d.obtenerP1() && p2 == d.obtenerP2() ;
    return ig;
}
```

En la clase *DispositivoFull*

- equals(d: Dispositivo): boolean*. Dos instancias de *Dispositivo* son equivalentes si tienen el mismo estado interno.
- riesgo(): boolean*. Retorna true si *p2* es mayor a *p1* o bien *p4* es mayor a *p3*.
- emergencia(): boolean*. Se considera en emergencia cuando *p4* es menor a *max*.



De modo que *equals* se *redefine* como sigue:

```
public boolean equals (Dispositivo d){
    boolean ig;
    DispositivoFull df = (DispositivoFull) d;
    ig = super.equals(d) && p3 == df.obtenerP3() && p4 == df.obtenerP4();
    return ig;
}
```

Identifique los errores que provoca el siguiente segmento de la clase tester y muestre la salida de las instrucciones que no provocan error:

```
Dispositivo s1 = new Dispositivo(1, 2);
Dispositivo s2 = new DispositivoFull(1, 2, 3, 2);
DispositivoFull s3 = new DispositivoFull(1, 2, 3, 4);
DispositivoFull s4 = new DispositivoFull(1, 2, 3, 1);
System.out.println(s1.equals(s2));
System.out.println(s1.equals(s3));
System.out.println(s2.equals(s1));
System.out.println(s2.equals(s3));
System.out.println(s3.equals(s1));
System.out.println(s3.equals(s2));
System.out.println(s3.equals(s4));
```

i) Analizando la salida del tester, ¿qué condiciones deben verificarse para que dos Dispositivos sean equivalentes, de acuerdo a este diseño e implementación?

j) Agregue la siguiente implementación para el método *equals* en la clase *DispositivoFull*:

```
public boolean equals(DispositivoFull e){
    boolean ig;
    if (this == e)
        ig = true;
    else if (e == null)
        ig = false;
    else if (getClass() != e.getClass())
        ig = false;
    else {
        DispositivoFull r = (DispositivoFull) e;
        ig = super.equals(e) && p3 == r.obtenerP3() && p4 == r.obtenerP4();
    }
    return ig;
}
```

k) Muestre la salida del segmento de código de la clase tester, luego de haber implementado *equals* en la clase *DispositivoFull*.



- l) Modifique el método *equals* en la clase *Dispositivo* y ejecute nuevamente el tester

```
public boolean equals( Dispositivo s ) {  
    boolean ig;  
    if (this == s)  
        ig = true;  
    else if (s == null)  
        ig = false;  
    else if (getClass() != s.getClass())  
        ig = false;  
    else {  
        ig = (p1 == s.p1) && (p2 == s.p2);  
    }  
    return ig;  
}
```

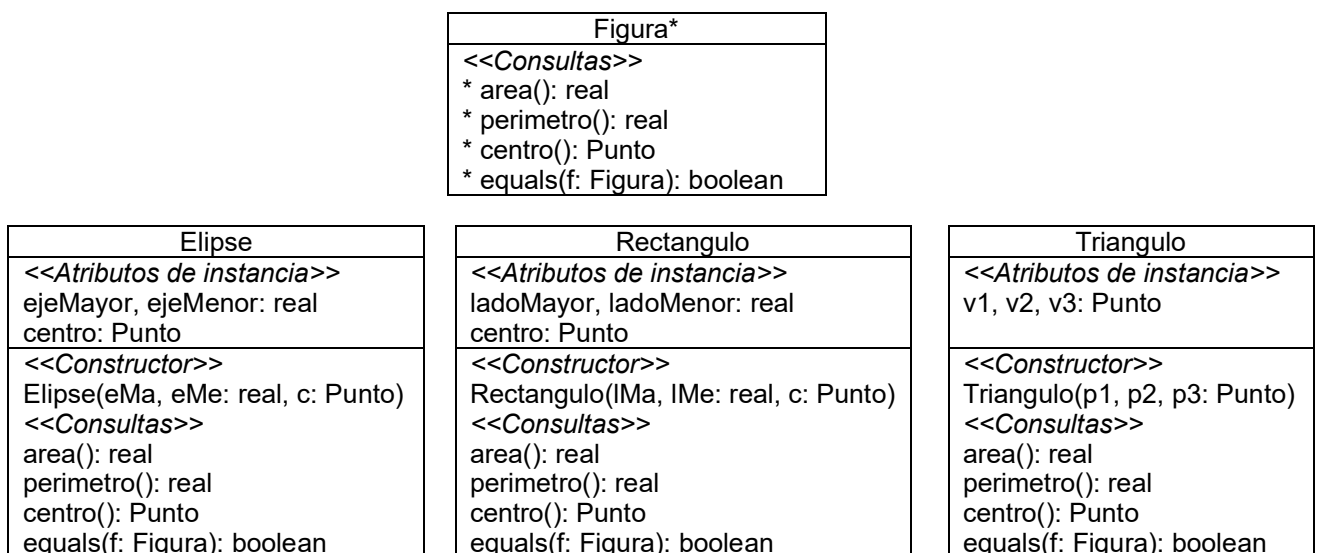
Observe que en esta última versión:

Dos dispositivos son equivalentes si son de la misma clase y tienen el mismo estado interno.

- m) Analice si se verifican las propiedades de la equivalencia (reflexiva, simétrica y transitiva) en cada diseño e implementación de *equals* en la clase *DispositivoFull*.
- n) Modifique la signatura del método *equals* de modo que quede sobrecargado en la clase *DispositivoFull* y vuelva a mostrar la salida al ejecutar el código del tester.

```
public boolean equals(DispositivoFull e)
```

EJERCICIO 14. Dada la clase *Punto* definida antes, implemente y verifique el siguiente diagrama en el cual *Elipse*, *Rectangulo* y *Triangulo* extienden a la clase abstracta *Figura*:



- El perímetro de la elipse se aproxima como $2\pi * \sqrt{((a^2+b^2)/2)}$ donde *a* y *b* son el semieje mayor (*mitad del eje*) y el semieje menor respectivamente.
- La superficie de la elipse se calcula como $\pi * \text{eje mayor} * \text{eje menor}$.



- El centro del triángulo es el promedio de sus vértices.
- El área del triángulo se calcula utilizando la *fórmula de Herón* a partir de su semiperímetro(s) y la longitud de sus lados (a, b y c).

$$\text{Área} = \sqrt{s(s-a)(s-b)(s-c)}$$

siendo a, b, c los tres lados y s el
semiperímetro $s = \frac{a+b+c}{2}$

- El método *equals* se implementa siempre en profundidad.

EJERCICIO 15.

1. Defina el concepto de clase abstracta y muestre un ejemplo ilustrativo de una clase abstracta que no tenga métodos abstractos.
2. Defina los conceptos de herencia, polimorfismo y vinculación dinámica. ¿Por qué la herencia favorece la productividad?
3. Defina los conceptos de sobrecarga y redefinición de métodos y relacione estos conceptos con la vinculación dinámica de código.
4. Defina los conceptos de variable polimórfica, asignación polimórfica y método polimórfico. ¿Qué restricciones impone Java sobre las asignaciones polimórficas?